

Prerequisites

From the first week of this course you **write, build and test programs on your own computer** — first in C for the midterm project, later in Java for the final project — and you keep your work under **Git and GitHub**. You therefore need to arrive with the knowledge and tools below. The formal prerequisites of the course are **CEN107 Algorithms and Programming I** and **CEN108 Algorithms and Programming II**; in particular, the development environment, Git, unit testing and template usage taught in the first weeks of CEN107 are **assumed** in this course.

⚠ Before the first class

Work through the "Test yourself before the first class" section near the end of this page on your own computer. If you get stuck on a step, go back to the corresponding CEN107/CEN108 material; if you still cannot solve it, ask in the first class.

1. What we bring from CEN107 and CEN108 (required)

Topic	Where is it taught?	Where is it used in this course?
Development environment: compiler (GCC/Clang/MSVC), IDE, WSL on Windows, building with CMake	CEN107 Week 2 — Development environments	Week 1's C workshop; setting up the midterm (C) project
Git and GitHub: creating a repository, <code>clone</code> , <code>commit</code> , <code>branches</code> , <code>pull request</code> , <code>.gitignore</code>	CEN107 Week 3 — Version management with Git	Project guide (fork, plan, submission) for both the midterm and the final checkpoint
Unit testing and coverage tools: writing tests, running them, reading a coverage report	CEN107 Week 4 — Unit testing and libraries	Midterm rubric criteria (GoogleTest, gcov/lcov); final rubric criteria (JUnit 5, JaCoCo)
Using project templates: forking a template, building it, producing its tests and documentation	CEN107 Weeks 2–4	Project guide — <code>cpp-cmake-ctest-template</code> for the midterm, <code>eclipse-java-maven-template</code> for the final
Basic algorithm analysis and recursion: counting operations, writing a simple recursive function	CEN108, early weeks	Big-O recap in Week 1; recursive solutions used from Week 3 onward (Towers of Hanoi, DFS, tree traversals, merge sort, quicksort)

2. C and Java programming fundamentals (required)

The midterm project is implemented in **C**, the final project in **Java**. Almost every data structure covered in this course is built directly out of the constructs below, so you should be able to read and write them comfortably before the first class.

2.1 C fundamentals (needed for the midterm project)

- pointers and pointer arithmetic (`int *p;` , `p + 1` , the difference between `p` and `*p`),
- arrays (declaration, indexing, passing an array to a function),
- `struct` definitions, and using a `struct` to group fields (for example a linked-list node),
- dynamic memory with `malloc` / `free` (and why every successful `malloc` needs a matching `free`),
- recursion (a function that calls itself, together with its base case),
- file input/output, including **binary files** (`fopen` with `"rb"` / `"wb"` , `fread` , `fwrite`).

2.2 Java fundamentals (needed for the final project)

- classes and objects (fields, constructors, methods, `this`),
- the basics of generics (using `List<T>` , declaring a simple generic class or method),
- exceptions (`try` / `catch` / `finally` , the difference between checked and unchecked exceptions).

3. Background briefly recalled in class

It is fine if you do not remember these in full detail; Week 1 gives a short recap before they are needed.

Background	Needed in which week?
Big-O notation basics: counting operations, comparing growth rates	Week 1, then used every week to compare data structures
Memory layout: stack vs. heap	Week 1, then referred back to whenever pointers and dynamic memory are used

4. What must be installed on your computer

A laptop is required. Come to the first class with these tools installed:

- A C compiler: **Visual Studio 2022** (Desktop development with C++) on Windows, or **GCC/Clang** on Linux/WSL/macOS
- **CMake** (the midterm project template is built and tested with it)
- A coverage tool for C: **gcov/lcov** on Linux/WSL, or **OpenCppCoverage** if you build with MSVC on Windows
- **Doxygen** (used to generate the project documentation)
- **JDK 21** and **Maven** (Maven pulls in JUnit 5 and JaCoCo automatically as project dependencies)
- **Git** and a **GitHub** account

Exact setup steps and the project templates themselves are given in the [project guide](#).

5. Test yourself before the first class

Try each task on your own computer before writing anything down, then compare your answer with the one given.

Task 1 — Compile and predict. What does the following program print?

```
#include <stdio.h>

int main(void) {
    int x = 5;
    int *p = &x;
    *p = *p + 1;
    printf("%d %d\n", x, *p);
    return 0;
}
```

✓ **Answer**

6 6

`p` holds the address of `x`, so `*p` and `x` name the same memory location; changing one changes the other.

Task 2 — Pointer arithmetic. Given `int arr[4] = {10, 20, 30, 40}; int *p = arr;`, what are the values of `*(p + 2)` and `p[2]`?

✓ **Answer**

Both are 30. `p[2]` is defined to mean `*(p + 2)`; array indexing is pointer arithmetic written with different syntax.

Task 3 — Find the bug. What is wrong with this function, and how would you fix it?

```
int *make_array(int n) {
    int arr[n];
    for (int i = 0; i < n; i++) arr[i] = i * i;
    return arr;
}
```

✓ **Answer**

`arr` is a local array that lives on the stack; it is destroyed the moment the function returns, so the pointer it returns is dangling. The fix is to allocate on the heap instead:

```
int *arr = malloc(n * sizeof(int));
```

and let the caller `free` it once it is no longer needed.

Task 4 — Struct and recursion. Define `struct Node { int value; struct Node *next; };` and write a recursive function `int length(struct Node *head)` that returns the number of nodes in the list. What does `length(NULL)` return, and why does the recursion stop there?

✓ Answer

```
int length(struct Node *head) {
    if (head == NULL) return 0;
    return 1 + length(head->next);
}
```

`length(NULL)` returns `0`; this is the base case, and without it the recursion would never stop.

Task 5 — Binary file I/O. After running this program, how many bytes does `data.bin` contain, and why?

```
#include <stdio.h>

int main(void) {
    int values[3] = {1, 2, 3};
    FILE *f = fopen("data.bin", "wb");
    fwrite(values, sizeof(int), 3, f);
    fclose(f);
    return 0;
}
```

✓ Answer

`3 * sizeof(int)` bytes — 12 bytes on almost every current desktop platform, since `sizeof(int) == 4` there. `fwrite` copies the raw bytes of the array into the file; unlike `fprintf`, there is no text formatting involved.

Task 6 — Java generics. What does this program print?

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        int sum = 0;
        for (int n : numbers) {
            sum += n;
        }
        System.out.println(sum);
    }
}
```

✓ Answer

30

`List<Integer>` is a generic collection restricted to `Integer` elements; the for-each loop reads every element and accumulates it into `sum`.

Task 7 — Java exceptions. In what order do the three letters print?

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            System.out.println("A");  
            throw new RuntimeException("boom");  
        } catch (RuntimeException e) {  
            System.out.println("B");  
        } finally {  
            System.out.println("C");  
        }  
    }  
}
```

✓ Answer

A
B
C

The exception thrown right after printing **A** is caught immediately by the matching `catch` block (**B**); the `finally` block always runs afterwards, whether or not an exception was thrown (**C**).

Task 8 — Build and test both toolchains. Run the checks below. Each command should finish without errors and print something close to the expected output.

```
gcc --version          # gcc (...) 11 or later – or clang/MSVC equivalent  
cmake --version        # cmake version 3.2x or later  
git clone https://github.com/<your-username>/cpp-cmake-ctest-template.git  
cd cpp-cmake-ctest-template  
cmake -S . -B build  
cmake --build build  
ctest --test-dir build  # expected: 100% tests passed  
  
java -version          # openjdk version "21..."  
mvn -version           # Apache Maven 3.9.x or later  
git clone https://github.com/<your-username>/eclipse-java-maven-template.git  
cd eclipse-java-maven-template  
mvn test               # expected: BUILD SUCCESS, Tests run: ..., Failures: 0
```

✓ What success looks like

Both toolchains report every test passing — `100% tests passed` for `ctest`, and `BUILD SUCCESS` with `Failures: 0` for Maven — with no compiler errors along the way. If either one fails, go back to Section 4, fix the missing tool, and try again before the first class.

If all eight tasks made sense and both toolchains built and tested cleanly, you are ready for the course. For the week-by-week schedule and the exact rubric for each checkpoint, see the [syllabus](#) and the [project guide](#).